第8章: CRUD実装（編集・削除・検索）

> この章で CRUD を完成させます。残りの **U（Update＝編集）** と **D（Delete＝削除）** を実装し、さらに使い勝手を上げる **検索機能** も追加します。第4章で学んだ動的ルート `[id].tsx`、第6章で作った `fetchBookById` / `updateBook` / `deleteBook` 関数を使います。

---

1. この章のゴール

- 一覧のカードを**タップすると編集画面へ移動**する
- 編集画面で**既存データを初期表示**し、変更して**更新（Update）** できる
- 編集画面から**削除（Delete）** できる（確認ダイアログ付き）
- 一覧画面に**検索ボックス**を追加し、タイトル・著者で絞り込む

---

2. 一覧カードをタップで編集画面へ

まず、第7章の `app/index.tsx` の一覧カードを「押せる」ようにし、押したら `/books/その本のid` へ移動するようにします。`renderItem` の `<View style={styles.card}>` を `<Pressable>` に変えます。

コピー

```
// app/index.tsx の renderItem を次のように変更する（変更点のみ抜粋）

renderItem={({ item }) => (
  // View → Pressable に変更。押すと編集画面へ移動する
  <Pressable
    style={styles.card}
    onPress={() => router.push(`/books/${item.id}`)}
    // `/books/${item.id}` : テンプレートリテラル。${item.id}にその本のidが埋め込まれる
    // 例: idが "abc" なら "/books/abc" というパスになり、books/[id].tsx が開く
  >
    <Text style={styles.title}>{item.title}</Text>
    <Text style={styles.author}>著者: {item.author}</Text>
    <View style={[styles.badge, getStatusStyle(item.status)]}>
      <Text style={styles.badgeText}>{item.status}</Text>
    </View>
  </Pressable>
)}
```

> **テンプレートリテラル `` `...${ }...` ``（復習）**：バッククォート `` ` `` で囲んだ文字列の中では、`${ }` を使って変数を埋め込めます。`` `/books/${item.id}` `` は「`/books/` に続けて、その本のidをつなげたパス」を作ります。ダブルクォートでの文字列連結（`"/books/" + item.id`）と同じ結果ですが、こちらの方が読みやすいです。
>
> **`Pressable` を import しているか確認**: 第7章で既に `import { ... Pressable ... } from "react-native";` に含まれています。もし無ければ追加してください。

---

3. 編集画面を作る（Update）

第6章で作った動的ルート `app/books/[id].tsx` を本実装します。この画面は「①idを受け取る → ②そのidの本をSupabaseから取得して初期表示 → ③変更して更新」という流れです。

> **▼ このコードがやること（先に日本語で）**：「一覧で押した本」を編集・削除できる画面を1枚作ります。まず `useLocalSearchParams` で「どの本を開いたか（id）」を受け取り、`useEffect` で画面表示時にその本のデータを取得して入力欄に初期表示します。あとは更新ボタンで `updateBook`、削除ボタンで（確認ダイアログ付きの）`deleteBook` を呼ぶだけです。これ1枚に CRUD の U（更新）と D（削除）が詰まっています。各行の細かい意味はコード内のコメントで説明します。

コピー

```
// app/books/[id].tsx – 編集画面（Update & Delete）

import { useState, useEffect } from "react";
import { View, Text, TextInput, Pressable, StyleSheet, ScrollView, Alert, ActivityIndicator } from "react-
native";
import { useLocalSearchParams, useRouter } from "expo-router";
// 第6章の関数（パスは ../../）
import { fetchBookById, updateBook, deleteBook } from "../../lib/books";

const STATUS_OPTIONS = ["未読", "読書中", "読了"];

export default function EditBookScreen() {
  const router = useRouter();
  // useLocalSearchParams : 現在のルートのパラメータを取得するフック
  // books/[id].tsx の [id] 部分を id という名前で受け取れる
  const { id } = useLocalSearchParams<{ id: string }>();

  // 入力欄のstate（第7章の登録フォームと同じ構成）
  const [title, setTitle] = useState("");
  const [author, setAuthor] = useState("");
  const [status, setStatus] = useState("未読");
  const [memo, setMemo] = useState("");
  // 初期データ読込中か
  const [loading, setLoading] = useState(true);
  // 更新処理中か
  const [saving, setSaving] = useState(false);

  // 画面表示時に、idの本を取得してフォームの初期値にセットする
  useEffect(() => {
    const load = async () => {
      try {
        // idで1冊取得（第6章）
        const book = await fetchBookById(id);
        // 取得した値を各stateにセット → フォームに初期表示される
        setTitle(book.title);
        setAuthor(book.author);
        setStatus(book.status);
        // memoがnullなら空文字に（?? で代替）
        setMemo(book.memo ?? "");
      } catch (e) {
        Alert.alert("読み込みエラー", "本の情報を取得できませんでした");
        console.log(e);
      } finally {
        setLoading(false);
      }
    };
    load();
  // 依存配列に id。idが変わったら取得し直す
  }, [id]);

  // 更新ボタンの処理
  const handleUpdate = async () => {
    if (title.trim() === "" || author.trim() === "") {
      Alert.alert("入力エラー", "タイトルと著者は必須です");
      return;
    }
    try {
      setSaving(true);
      // updateBook(id, 新しい内容) : 第6章の更新関数。idの本を上書きする
      await updateBook(id, {
        title: title.trim(),
        author: author.trim(),
```

```
      status: status,
      memo: memo.trim() === "" ? null : memo.trim(),
    });
    // 一覧へ戻る（戻り先で再取得され反映）
    router.back();
  } catch (e) {
    Alert.alert("更新に失敗しました", "通信状況を確認してください");
    console.log(e);
  } finally {
    setSaving(false);
  }
};

// 削除ボタンの処理（確認ダイアログを出す）
const handleDelete = () => {
  // Alert.alert(タイトル, 本文, ボタン配列) : ボタンを複数置けるダイアログ
  Alert.alert(
    "削除の確認",
    "この本を削除しますか？この操作は取り消せません。",
    [
      // style:"cancel" : キャンセル用のボタン（何もしない）
      { text: "キャンセル", style: "cancel" },
      {
        text: "削除",
        // style:"destructive" : 赤字の警告ボタン（iOS）
        style: "destructive",
        // 「削除」を押したときだけ実行
        onPress: async () => {
          try {
            // 第6章の削除関数
            await deleteBook(id);
            // 一覧へ戻る
            router.back();
          } catch (e) {
            Alert.alert("削除に失敗しました", "通信状況を確認してください");
            console.log(e);
          }
        },
      },
    ]
  );
};

// 初期データ読込中はぐるぐるを表示
if (loading) {
  return (
    <View style={styles.center}>
      <ActivityIndicator size="large" color="#1e40af" />
    </View>
  );
}

return (
  <ScrollView style={styles.container} contentContainerStyle={{ padding: 20, gap: 18 }}>
    {/* タイトル */}
    <View>
      <Text style={styles.label}>タイトル *</Text>
      <TextInput style={styles.input} value={title} onChangeText={setTitle} placeholder="タイトル" />
    </View>
    {/* 著者 */}
    <View>
      <Text style={styles.label}>著者 *</Text>
      <TextInput style={styles.input} value={author} onChangeText={setAuthor} placeholder="著者" />
    </View>
    {/* ステータス */}
```

```
      <View>
        <Text style={styles.label}>ステータス</Text>
        <View style={styles.statusRow}>
          {STATUS_OPTIONS.map((option) => (
            <Pressable
              key={option}
              style={[styles.statusButton, status === option && styles.statusButtonActive]}
              onPress={() => setStatus(option)}
            >
              <Text style={[styles.statusText, status === option && styles.statusTextActive]}>{option}
</Text>
            </Pressable>
          ))}
        </View>
      </View>
      {/* メモ */}
      <View>
        <Text style={styles.label}>メモ</Text>
        <TextInput
          style={[styles.input, styles.textarea]}
          value={memo} onChangeText={setMemo}
          placeholder="感想や覚え書き" multiline numberOfLines={4}
        />
      </View>

      {/* 更新ボタン */}
      <Pressable
        style={[styles.saveButton, saving && styles.saveButtonDisabled]}
        onPress={handleUpdate} disabled={saving}
      >
        <Text style={styles.saveButtonText}>{saving ? "更新中..." : "更新する"}</Text>
      </Pressable>

      {/* 削除ボタン（赤） */}
      <Pressable style={styles.deleteButton} onPress={handleDelete}>
        <Text style={styles.deleteButtonText}>この本を削除</Text>
      </Pressable>
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: "#f8fafc" },
  center: { flex: 1, justifyContent: "center", alignItems: "center" },
  label: { fontSize: 13, fontWeight: "600", color: "#334155", marginBottom: 6 },
  input: {
    backgroundColor: "#fff", borderWidth: 1, borderColor: "#e2e8f0", borderRadius: 8,
    paddingHorizontal: 12, paddingVertical: 10, fontSize: 14,
  },
  textarea: { height: 100, textAlignVertical: "top" },
  statusRow: { flexDirection: "row", gap: 8 },
  statusButton: {
    flex: 1, alignItems: "center", paddingVertical: 10,
    borderWidth: 1, borderColor: "#e2e8f0", borderRadius: 8, backgroundColor: "#fff",
  },
  statusButtonActive: { borderColor: "#1e40af", backgroundColor: "#eff6ff" },
  statusText: { fontSize: 13, color: "#475569" },
  statusTextActive: { color: "#1e40af", fontWeight: "700" },
  saveButton: { backgroundColor: "#1e40af", paddingVertical: 14, borderRadius: 10, alignItems: "center",
marginTop: 8 },
  saveButtonDisabled: { backgroundColor: "#94a3b8" },
  saveButtonText: { color: "#fff", fontWeight: "bold", fontSize: 15 },
  deleteButton: { paddingVertical: 14, borderRadius: 10, alignItems: "center", borderWidth: 1, borderColor:
"#fecaca", backgroundColor: "#fef2f2" },
```

```
  deleteButtonText: { color: "#dc2626", fontWeight: "bold", fontSize: 14 },
});
```

3.1 重要ポイントの解説

① **`useLocalSearchParams` でidを受け取る** 動的ルート `books/[id].tsx` の `[id]` 部分の値（どの本を開いたか）を、`useLocalSearchParams` で取得します。`const { id } = useLocalSearchParams<{ id: string }>()` で、`/books/abc` を開いたなら `id` に `"abc"` が入ります。

> **`useLocalSearchParams<{ id: string }>()` の山カッコ:** `<{ id: string }>` は「受け取るパラメータの型」をTypeScriptに教えています。`books/[id].tsx` なので `id` という文字列が来る、という宣言です。これで `id` を安全に使えます。

② **既存データを初期表示する流れ** `useEffect`（画面表示時に1回）で `fetchBookById(id)` を呼び、取得した本の各値を `setTitle` などでstateに入れます。stateが入力欄の `value` に結びついているので、**フォームに既存の値が最初から表示**されます。これが「編集」の体験を作る肝です。

③ **削除の確認ダイアログ** 削除は取り消せない操作なので、いきなり消さず `Alert.alert` **の第3引数にボタン配列**を渡して確認します。「キャンセル」と「削除」の2つを置き、「削除」を押したときだけ実際に `deleteBook` を実行します。`style: "destructive"` は赤い警告色のボタン（iOS）になります。

> **なぜ確認を挟む？**「うっかり削除」を防ぐためです。データを失う・お金が動く・外部に送信する、といった**取り返しのつかない操作**の前には、必ずユーザーに確認を取るのが良いアプリ設計です。

▼ コードを1つずつ分解して解説

上の編集画面には、初心者がつまずきやすい書き方がいくつも入っています。順番に、**塊（かたまり）ごと**にていねいに見ていきましょう。今すぐ全部を暗記する必要はありません。「この塊は何をしているか」が分かれば十分です。

解説1: 画面の準備（import と id の受け取り）

コピー

```
import { useState, useEffect } from "react";
import { View, Text, TextInput, Pressable, StyleSheet, ScrollView, Alert, ActivityIndicator } from "react-native";
import { useLocalSearchParams, useRouter } from "expo-router";
// 第6章の関数（パスは ../../）
import { fetchBookById, updateBook, deleteBook } from "../../lib/books";

const STATUS_OPTIONS = ["未読", "読書中", "読了"];

export default function EditBookScreen() {
  const router = useRouter();
  // useLocalSearchParams : 現在のルートのパラメータを取得するフック
  // books/[id].tsx の [id] 部分を id という名前で受け取れる
  const { id } = useLocalSearchParams<{ id: string }>();
```

- `import { ... } from "..."` … この画面で使う部品を読み込んでいます。`react-native` からは画面に置く部品（`View`＝箱、`Text`＝文字、`TextInput`＝入力欄、`Pressable`＝押せる領域、`Alert`＝ダイアログ、`ActivityIndicator`＝ぐるぐる）を取り込みます。
- `import { fetchBookById, updateBook, deleteBook } from "../../lib/books"` … 第6章で作った「1冊取得・更新・削除」の3つの関数を読み込みます。`../../` は「2つ上のフォルダ」を意味し、`app/books/[id].tsx` から `lib/books` までさかのぼる道順です。
- `const STATUS_OPTIONS = [...]` … ステータスの選択肢（未読・読書中・読了）を1か所にまとめた配列です。あとでボタンを並べるときにこの配列を使い回します。
- `useRouter()` … 画面遷移（戻る・進む）を行うための道具を受け取ります。更新・削除のあと一覧へ戻るのに使います。
- `const { id } = useLocalSearchParams<{ id: string }>()` … 「どの本を開いたか」を表す `id` を受け取ります。`/books/abc` を開いたなら `id` に `"abc"` が入ります。

> **用語: フック（Hook）** … `use` で始まる関数の総称です（`useState`・`useEffect`・`useRouter` など）。React の機能（状態を持つ・画面表示時に処理する・画面遷移する等）をコンポーネントから使うための「専用の道具」だと考えてください。必ずコンポーネント関数の**先頭**で呼ぶ、というルールがあります。

解説2: 入力欄と画面状態を覚えておく state

コピー

```
// 入力欄のstate（第7章の登録フォームと同じ構成）
const [title, setTitle] = useState("");
const [author, setAuthor] = useState("");
const [status, setStatus] = useState("未読");
const [memo, setMemo] = useState("");
// 初期データ読込中か
const [loading, setLoading] = useState(true);
// 更新処理中か
const [saving, setSaving] = useState(false);
```

- `const [今の値, 値を変える関数] = useState(初期値)` という形で、**変化する値**を1つずつ用意しています。
- `title / author / status / memo` … それぞれの入力欄の中身を覚えておく箱です。あとで入力欄の `value` に結びつけるので、ここが画面の表示そのものになります。`status` だけは初期値を `"未読"` にしています。
- `loading` … 「最初のデータを読み込んでいる途中か」を表す `true` / `false`。最初は `true`（読み込み中）にしておき、取得が終わったら `false` にしてフォームを表示します。
- `saving` … 「更新ボタンを押して保存中か」を表します。保存中はボタンを押せないようにして、二重送信を防ぐのに使います。

> **用語: state（ステート＝状態）** … コンポーネントが内部で覚えておく「変化する値」のことです。`setXxx(...)` で値を変えると、React がその箇所を自動で描き直してくれます。**直接書き換えず必ず `setXxx` を使う**のが鉄則です。

解説3: 画面表示時に既存データを取りに行く `useEffect`

コピー

```
// 画面表示時に、idの本を取得してフォームの初期値にセットする
useEffect(() => {
  const load = async () => {
    try {
      // idで1冊取得（第6章）
      const book = await fetchBookById(id);
      // 取得した値を各stateにセット → フォームに初期表示される
      setTitle(book.title);
      setAuthor(book.author);
      setStatus(book.status);
      // memoがnullなら空文字に（?? で代替）
      setMemo(book.memo ?? "");
    } catch (e) {
      Alert.alert("読み込みエラー", "本の情報を取得できませんでした");
      console.log(e);
    } finally {
      setLoading(false);
    }
  };
  load();
// 依存配列に id。idが変わったら取得し直す
}, [id]);
```

- **`useEffect(() => { ... }, [id])`** … 「画面が表示されたとき（と `id` が変わったとき）に1回だけ実行する処理」を書く場所です。データの取得など「画面描画以外の裏方の仕事」をここで行います。
- **`const load = async () => { ... }`** … 取得処理を `load` という関数にまとめています。`async` は「中で `await`（待つ）を使う関数」という目印です。通信は時間がかかるので「待つ」必要があります。
- **`const book = await fetchBookById(id)`** … `id` の本をSupabaseから取りに行き、返ってくるまで `await` で待ちます。取れたら `setTitle(book.title)` などで各stateに入れ、**入力欄に既存の値が初期表示**されます。これが「編集画面」の体験を作る肝です。
- **`book.memo ?? ""`** … `??`（ナル合体演算子）は「左が `null` / `undefined` のときだけ右を使う」記号です。メモが未入力（`null`）なら空文字 `""` にしています。
- **`try / catch / finally`** … 通信は失敗することがあるため、`try` で正常時、`catch` で失敗時（エラーダイアログ表示）、`finally` で「成功・失敗どちらでも最後に必ず」`setLoading(false)` を実行し、ぐるぐる表示を終えます。

> **用語: 依存配列（dependency array）** … `useEffect` の第2引数 `[id]` のことです。「この配列の中の値が変わったら、もう一度この処理を実行する」という指定です。`[]`（空）なら最初の1回だけ、`[id]` なら `id` が変わるたびに実行されます。

解説4: 更新ボタンの処理 `handleUpdate`

コピー

```
// 更新ボタンの処理
const handleUpdate = async () => {
  if (title.trim() === "" || author.trim() === "") {
    Alert.alert("入力エラー", "タイトルと著者は必須です");
    return;
  }
  try {
    setSaving(true);
    // updateBook(id, 新しい内容) : 第6章の更新関数。idの本を上書きする
    await updateBook(id, {
      title: title.trim(),
      author: author.trim(),
      status: status,
      memo: memo.trim() === "" ? null : memo.trim(),
    });
    // 一覧へ戻る（戻り先で再取得され反映）
    router.back();
  } catch (e) {
    Alert.alert("更新に失敗しました", "通信状況を確認してください");
    console.log(e);
  } finally {
    setSaving(false);
  }
};
```

- `if (title.trim() === "" || author.trim() === "")` … 送信前の**入力チェック**です。`trim()` は前後の空白を取り除く処理で、空白だけの入力も「未入力」とみなします。タイトルか著者が空ならエラーを出し、`return` で処理を中断します。
- `setSaving(true)` … 保存処理の開始を記録します。これでボタンが「更新中…」表示になり、押せなくなります（二重送信防止）。
- `await updateBook(id, { ... })` … `id` の本を、入力された新しい内容で上書きします。`memo.trim() === "" ? null : memo.trim()` は三項演算子で、「メモが空なら `null`、入力があればその値」を渡しています。
- `router.back()` … 更新が終わったら一覧画面へ戻ります。戻った先でデータが再取得され、変更が反映されます。

- `finally { setSaving(false) }` … 成功・失敗どちらでも最後に保存状態を解除し、ボタンを元に戻します。

> **用語: 二重送信（ダブルサブミット）** … 通信中にユーザーがボタンを連打して、同じ更新が何度も送られてしまう不具合です。`saving` を `true` にしてボタンを無効化することで防ぎます。

解説5: 削除ボタンと確認ダイアログ `handleDelete`

コピー

```
// 削除ボタンの処理（確認ダイアログを出す）
const handleDelete = () => {
  // Alert.alert(タイトル, 本文, ボタン配列) : ボタンを複数置けるダイアログ
  Alert.alert(
    "削除の確認",
    "この本を削除しますか？この操作は取り消せません。",
    [
      // style:"cancel" : キャンセル用のボタン（何もしない）
      { text: "キャンセル", style: "cancel" },
      {
        text: "削除",
        // style:"destructive" : 赤字の警告ボタン（iOS）
        style: "destructive",
        // 「削除」を押したときだけ実行
        onPress: async () => {
          try {
            // 第6章の削除関数
            await deleteBook(id);
            // 一覧へ戻る
            router.back();
          } catch (e) {
            Alert.alert("削除に失敗しました", "通信状況を確認してください");
            console.log(e);
          }
        },
      },
    ]
  );
};
```

- `Alert.alert(タイトル, 本文, ボタン配列)` … 確認ダイアログを表示します。第3引数に**ボタンの配列**を渡すと、複数のボタンを置けます。
- `{ text: "キャンセル", style: "cancel" }` …「キャンセル」ボタンです。`onPress` を書いていないので、押しても何も起きずダイアログが閉じるだけです。
- `style: "destructive"` …「削除」ボタンを赤い警告色（iOS）にします。取り消せない危険な操作だと見た目で伝えます。

- `onPress: async () => { ... }` … 「削除」を押したときだけ実行される処理です。中で `await deleteBook(id)` を呼び、消えたら `router.back()` で一覧へ戻ります。
- 削除を「いきなり実行せず、必ず確認を1段はさむ」ことで、うっかり削除を防いでいます。

> **用語: 確認ダイアログ（confirmation dialog）** … 重要な操作の前に「本当に実行してよいか」をユーザーに尋ねる小窓のことです。削除・送信・購入など、取り返しのつかない操作の前に置くのが良い設計です。

解説6: 読込中のぐるぐる表示（早期 return）

コピー

```
// 初期データ読込中はぐるぐるを表示
if (loading) {
  return (
    <View style={styles.center}>
      <ActivityIndicator size="large" color="#1e40af" />
    </View>
  );
}
```

- `if (loading) { return ... }` … `loading` が `true`（まだデータを取得中）のあいだは、フォームの代わりに**ぐるぐる（読み込み中アイコン）だけ**を表示して、ここで `return` します。これを「早期 return」と呼びます。
- `<ActivityIndicator />` … くるくる回る読み込み中アイコンです。`size="large"` で大きめ、`color` で色を指定しています。
- 取得が終わると（解説3の `setLoading(false)` により）`loading` が `false` になり、この `if` を素通りして、下の本来のフォームが表示されます。

> **用語: 早期 return（early return）** … 関数の途中で条件を満たしたら、その時点で `return` して残りの処理を実行しない書き方です。「読み込み中ならフォームを描かずぐるぐるだけ返す」のように、画面を切り替えるのに便利です。

解説7: 入力フォームの本体（JSX）

コピー

```
return (
  <ScrollView style={styles.container} contentContainerStyle={{ padding: 20, gap: 18 }}>
    {/* タイトル */}
    <View>
      <Text style={styles.label}>タイトル *</Text>
      <TextInput style={styles.input} value={title} onChangeText={setTitle} placeholder="タイトル" />
    </View>
    {/* 著者 */}
    <View>
      <Text style={styles.label}>著者 *</Text>
      <TextInput style={styles.input} value={author} onChangeText={setAuthor} placeholder="著者" />
    </View>
    {/* ステータス */}
    <View>
      <Text style={styles.label}>ステータス</Text>
      <View style={styles.statusRow}>
        {STATUS_OPTIONS.map((option) => (
          <Pressable
            key={option}
            style={[styles.statusButton, status === option && styles.statusButtonActive]}
            onPress={() => setStatus(option)}
          >
            <Text style={[styles.statusText, status === option && styles.statusTextActive]}>{option}</Text>
          </Pressable>
        ))}
      </View>
    </View>
    {/* メモ */}
    <View>
      <Text style={styles.label}>メモ</Text>
      <TextInput
        style={[styles.input, styles.textarea]}
        value={memo} onChangeText={setMemo}
        placeholder="感想や覚え書き" multiline numberOfLines={4}
      />
    </View>
```

- `<ScrollView>` … 中身が画面より長くなってもスクロールして見られる箱です。`contentContainerStyle` の `gap: 18` で各入力欄のあいだに均等な隙間を作っています。
- `<TextInput value={title} onChangeText={setTitle} />` … 入力欄です。`value={title}` で「stateの値を表示」し、`onChangeText={setTitle}` で「打つたびに state を更新」します。この2つがペアになって、**stateと画面が常に一致**します。
- `{STATUS_OPTIONS.map((option) => (...))}` … ステータスの選択肢を `map` でボタンに展開しています。`key={option}` はReactがリストの各項目を見分けるための目印です。
- `status === option && styles.statusButtonActive` …「いま選んでいるステータスのボタンだけ」見た目を変える書き方です。`A && B` は「Aが真ならBを使う」ので、選択中のボタンにだけ強調スタイルが付きます。
- `onPress={() => setStatus(option)}` … ボタンを押すと、そのステータスを `status` にセットします。`multiline numberOfLines={4}` はメモ欄を複数行入力にする指定です。

> **用語: 制御コンポーネント（controlled component）** … `value`（表示）と `onChangeText`（更新）の両方をstateに結びつけ、「入力欄の中身を完全にstateで管理する」入力欄のことです。Reactの入力フォームの基本形です。

解説8: 更新ボタン・削除ボタンの JSX

コピー

```
      {/* 更新ボタン */}
      <Pressable
        style={[styles.saveButton, saving && styles.saveButtonDisabled]}
        onPress={handleUpdate} disabled={saving}
      >
        <Text style={styles.saveButtonText}>{saving ? "更新中..." : "更新する"}</Text>
      </Pressable>

      {/* 削除ボタン（赤） */}
      <Pressable style={styles.deleteButton} onPress={handleDelete}>
        <Text style={styles.deleteButtonText}>この本を削除</Text>
      </Pressable>
    </ScrollView>
  );
}
```

- **`onPress={handleUpdate}`** … 更新ボタンを押すと、解説4の `handleUpdate` が呼ばれます。関数名だけを渡す（`()` を付けない）のがポイントで、`()` を付けると押す前に即実行されてしまいます。
- **`disabled={saving}`** … 保存中（`saving` が `true`）はボタンを無効化し、連打を防ぎます。同時に `saving && styles.saveButtonDisabled` でボタンの色を薄くして「今は押せない」と見た目でも伝えます。
- **`{saving ? "更新中..." : "更新する"}`** … 三項演算子で、保存中はラベルを「更新中...」に切り替えます。
- **`<Pressable ... onPress={handleDelete}>`** … 削除ボタンは解説5の `handleDelete` を呼びます。赤系のスタイル（`deleteButton`）で危険な操作だと示しています。

> **用語: `disabled`（無効化）** … ボタンを「押せない状態」にする指定です。処理中や入力が不十分なときに `true` にして、誤操作・二重操作を防ぎます。

解説9: 見た目を決める styles

コピー

```
const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: "#f8fafc" },
  center: { flex: 1, justifyContent: "center", alignItems: "center" },
  label: { fontSize: 13, fontWeight: "600", color: "#334155", marginBottom: 6 },
  input: {
    backgroundColor: "#fff", borderWidth: 1, borderColor: "#e2e8f0", borderRadius: 8,
    paddingHorizontal: 12, paddingVertical: 10, fontSize: 14,
  },
  textarea: { height: 100, textAlignVertical: "top" },
  statusRow: { flexDirection: "row", gap: 8 },
  statusButton: {
    flex: 1, alignItems: "center", paddingVertical: 10,
    borderWidth: 1, borderColor: "#e2e8f0", borderRadius: 8, backgroundColor: "#fff",
  },
  statusButtonActive: { borderColor: "#1e40af", backgroundColor: "#eff6ff" },
  statusText: { fontSize: 13, color: "#475569" },
  statusTextActive: { color: "#1e40af", fontWeight: "700" },
  saveButton: { backgroundColor: "#1e40af", paddingVertical: 14, borderRadius: 10, alignItems: "center",
marginTop: 8 },
  saveButtonDisabled: { backgroundColor: "#94a3b8" },
  saveButtonText: { color: "#fff", fontWeight: "bold", fontSize: 15 },
  deleteButton: { paddingVertical: 14, borderRadius: 10, alignItems: "center", borderWidth: 1, borderColor:
"#fecaca", backgroundColor: "#fef2f2" },
  deleteButtonText: { color: "#dc2626", fontWeight: "bold", fontSize: 14 },
});
```

- `StyleSheet.create({ ... })` … 見た目（色・余白・大きさ）のまとまりを作る関数です。ここで作った名前を、JSX側で `style={styles.input}` のように呼び出します。
- `flex: 1` …「使える空間いっぱいに広がる」指定です。`container` や `center` を画面全体に広げるのに使います。
- `statusButtonActive` / `saveButtonDisabled` … 状態に応じて切り替える「追加用」のスタイルです。選択中・無効中のときだけ、基本スタイルに重ねて適用します。
- 色は `#1e40af`（濃い青＝主役の色）、`#dc2626`（赤＝削除の警告色）のように、役割ごとに決めています。

> **用語: `StyleSheet`** … React Native でスタイルをまとめて定義する仕組みです。Webの CSS に相当しますが、キーはキャメルケース（ `background-color` → `backgroundColor` ）で書き、値は基本的に数値（単位なし）や文字列で指定します。

---

4. 検索機能を追加する

最後に、一覧画面に検索ボックスを付け、タイトル・著者で絞り込めるようにします。`app/index.tsx` を改良します。検索は「**取得済みのデータをアプリ側で絞り込む**」方式にします（シンプルで高速）。

▼ **このコードがやること（先に日本語で）**：一覧画面に検索ボックスを足し、入力した言葉でタイトル・著者を絞り込んで表示します。仕組みはシンプルで、検索キーワードを `keyword` という state に入れ、`useMemo` の中で `filter` を使って「キーワードを含む本だけ」を残した配列を作り、それを `FlatList` に渡すだけです。サーバーに問い合わせ直すのではなく、すでに取得済みの一覧をアプリ側で絞り込むのがポイントです。`useMemo` や `includes` の意味はコード内のコメントとこの後の解説で説明します。

コピー

```
// app/index.tsx に検索機能を追加する（主な変更点）

// useMemo を追加
import { useState, useCallback, useMemo } from "react";
// TextInput を追加
import { TextInput } from "react-native";
//  （他のimportは第7章のまま）

export default function HomeScreen() {
  const router = useRouter();
  const [books, setBooks] = useState<Book[]>([]);
  const [loading, setLoading] = useState(true);
  // ★追加: 検索キーワードのstate
  const [keyword, setKeyword] = useState("");

  //  （load関数・useFocusEffectは第7章のまま）

  // ★追加: キーワードで絞り込んだ結果を計算する
  // useMemo : 計算結果を覚えておき、必要なときだけ再計算するフック（無駄な計算を減らす）
  const filteredBooks = useMemo(() => {
    // 未入力なら全件をそのまま返す
    if (keyword.trim() === "") return books;
    // 大文字小文字を区別しないため小文字化
    const lower = keyword.toLowerCase();
    // filter : 条件に合う本だけ残す（第2章）。タイトルか著者にキーワードを含むものを抽出
    return books.filter(
      (b) =>
        // includes : 文字列に含まれるか判定 / || は「または」
        b.title.toLowerCase().includes(lower) ||
        b.author.toLowerCase().includes(lower)
    );
  // booksかkeywordが変わったときだけ再計算
  }, [books, keyword]);

  if (loading) {
    return (
      <View style={styles.center}>
        <ActivityIndicator size="large" color="#1e40af" />
      </View>
    );
  }

  return (
    <View style={styles.container}>
      {/* ★追加: 検索ボックス */}
      <View style={styles.searchWrap}>
        <TextInput
          style={styles.search}
          placeholder="🔍 タイトルや著者で検索..."
          value={keyword}
          onChangeText={setKeyword}
        />
      </View>

      <FlatList
        // ★変更: books → filteredBooks（絞り込み後を表示）
        data={filteredBooks}
        keyExtractor={(item) => item.id}
        contentContainerStyle={{ padding: 16, gap: 10 }}
        ListEmptyComponent={
          // 検索中かどうかでメッセージを出し分ける（三項演算子）
          <Text style={styles.empty}>
```

```
            {keyword.trim() === "" ? "まだ本が登録されていません。" : "該当する本が見つかりません。"}
          </Text>
        }
        renderItem={({ item }) => (
          <Pressable style={styles.card} onPress={() => router.push(`/books/${item.id}`)}>
            <Text style={styles.title}>{item.title}</Text>
            <Text style={styles.author}>著者: {item.author}</Text>
            <View style={[styles.badge, getStatusStyle(item.status)]}>
              <Text style={styles.badgeText}>{item.status}</Text>
            </View>
          </Pressable>
        )}
      />

      <Pressable style={styles.fab} onPress={() => router.push("/new")}>
        <Text style={styles.fabText}>＋</Text>
      </Pressable>
    </View>
  );
}

// styles に検索ボックス用を追加
// searchWrap: { padding: 16, paddingBottom: 0 },
// search: { backgroundColor: "#fff", borderWidth: 1, borderColor: "#e2e8f0", borderRadius: 8,
paddingHorizontal: 14, paddingVertical: 10, fontSize: 14 },
```

> **useMemo とは？** useMemo （ユーズ・メモ）は「重い計算の結果を覚えておき、関係する値が変わったときだけ再計算する」フックです。ここでは「キーワードでの絞り込み」を、books か keyword が変わったときだけ実行します。本が少ないうちは無くても動きますが、データが増えても軽快に保つための良い習慣です。
>
> **includes と toLowerCase**：- 文字列.includes("探す文字") … その文字が含まれていれば true 。- 文字列.toLowerCase() … すべて小文字に変換。「ABC」で検索しても「abc」がヒットするよう、両方を小文字に揃えて比較しています。
>
> **サーバー側で検索する方法（発展）**：本書はアプリ側で絞り込みましたが、データが非常に多い場合は Supabase 側で .ilike("title", \ %${keyword}%`) のように検索させる方法もあります（ ilike` は大文字小文字を無視した部分一致）。まずはアプリ側の方式で十分です。

▼ コードを1つずつ分解して解説

検索機能の追加分も、塊ごとに見ていきましょう。ポイントは「**キーワードを覚える state**」と「**絞り込み結果を計算する `useMemo`**」の2つです。

解説1: 検索キーワードを覚える state

コピー

```
// ★追加: 検索キーワードのstate
const [keyword, setKeyword] = useState("");
```

- 検索ボックスに入力された言葉を覚えておくための state です。初期値は空文字 `""`（＝何も入力されていない状態）。
- `keyword` が「今の検索語」、`setKeyword` が「検索語を更新する関数」です。あとで検索ボックスの `value` と `onChangeText` に結びつけ、入力するたびに更新します。
- この値が変わるたびに、次の解説2の絞り込みが再計算され、画面の一覧が自動で変わります。

> **用語: 検索キーワード** … ユーザーが絞り込みのために打ち込む言葉のことです。ここではこの1つの値を起点に、表示する本のリストが決まります。

解説2: キーワードで絞り込む `useMemo`

コピー

```
// ★追加: キーワードで絞り込んだ結果を計算する
// useMemo : 計算結果を覚えておき、必要なときだけ再計算するフック（無駄な計算を減らす）
const filteredBooks = useMemo(() => {
  // 未入力なら全件をそのまま返す
  if (keyword.trim() === "") return books;
  // 大文字小文字を区別しないため小文字化
  const lower = keyword.toLowerCase();
  // filter : 条件に合う本だけ残す（第2章）。タイトルか著者にキーワードを含むものを抽出
  return books.filter(
    (b) =>
      // includes : 文字列に含まれるか判定 / || は「または」
      b.title.toLowerCase().includes(lower) ||
      b.author.toLowerCase().includes(lower)
  );
// booksかkeywordが変わったときだけ再計算
}, [books, keyword]);
```

- `useMemo(() => { ... }, [books, keyword])` … 中の計算結果（＝絞り込んだ配列）を覚えておき、`books` か `keyword` が変わったときだけ再計算するフックです。無駄な計算を減らして動作を軽快に保ちます。
- `if (keyword.trim() === "") return books` … 検索語が空なら、絞り込まずに全件をそのまま返します。
- `const lower = keyword.toLowerCase()` … 検索語を小文字に変換します。比較する側も小文字にそろえることで、「ABC」で検索しても「abc」がヒットするようにします（大文字小文字を区別しない）。
- `books.filter((b) => ...)` … 条件に合う本だけを残した新しい配列を作ります。条件は「タイトル**または**著者に検索語を含む」で、`||` が「または」を表します。
- `.includes(lower)` … その文字列に検索語が含まれていれば `true` を返すメソッドです。タイトル・著者それぞれで判定しています。

> **用語: `useMemo` （ユーズ・メモ）** … 「重い計算の結果を覚えておき、関係する値（第2引数の配列）が変わったときだけ計算し直す」フックです。同じ入力なら前回の結果を使い回すことで、再描画のたびに無駄な計算をしないようにします。

解説3: 検索ボックスと絞り込み結果の表示（JSX）

コピー

```
      {/* ★追加: 検索ボックス */}
      <View style={styles.searchWrap}>
        <TextInput
          style={styles.search}
          placeholder="🔍 タイトルや著者で検索..."
          value={keyword}
          onChangeText={setKeyword}
        />
      </View>

      <FlatList
        // ★変更: books → filteredBooks（絞り込み後を表示）
        data={filteredBooks}
        keyExtractor={(item) => item.id}
        contentContainerStyle={{ padding: 16, gap: 10 }}
        ListEmptyComponent={
          // 検索中かどうかでメッセージを出し分ける（三項演算子）
          <Text style={styles.empty}>
            {keyword.trim() === "" ? "まだ本が登録されていません。" : "該当する本が見つかりません。"}
          </Text>
        }
```

- **`<TextInput value={keyword} onChangeText={setKeyword} />`** … 検索ボックスです。`value={keyword}` で state の値を表示し、`onChangeText={setKeyword}` で打つたびに state を更新します。これで入力が即座に絞り込みに反映されます。
- **`data={filteredBooks}`** … `FlatList`（一覧）に渡すデータを、全件の `books` ではなく**絞り込み後の `filteredBooks`** に変えています。これが検索結果の表示そのものです。
- **`ListEmptyComponent`** … 表示する本が0件のときに出すメッセージです。三項演算子で「まだ未登録（検索語が空）」と「該当なし（検索語あり）」のメッセージを出し分けています。

> **用語: `placeholder` （プレースホルダー）** … 入力欄が空のときに薄く表示される案内文です。ここでは「🔍 タイトルや著者で検索...」と表示し、何を入力すればよいかをユーザーに伝えています。

---

5. 動作確認 — CRUD完成！

`npx expo start` で起動し、すべての機能を通しで確認しましょう。

操作	期待する動作
一覧のカードをタップ	編集画面が開き、その本の情報が初期表示される
内容を変えて「更新する」	一覧に戻り、変更が反映されている
「この本を削除」→「削除」	確認後に削除され、一覧から消える
「削除」確認で「キャンセル」	何も起きない（消えない）
検索ボックスに文字入力	タイトル・著者で絞り込まれる
該当なしの語で検索	「該当する本が見つかりません」と表示

これで **CRUD（作成・読取・更新・削除）＋検索** がすべて揃いました。アプリとして「使える」状態になっています！🎉

> **Supabase側でも確認:** Supabaseダッシュボードの「Table Editor」で `books` テーブルを開くと、アプリでの追加・更新・削除がデータベースに反映されているのが分かります。アプリとデータベースが本当につながっている実感が得られます。

---

6. この章のまとめ

- 一覧カードを `Pressable` にし、`router.push(`/books/${item.id}`)` で編集画面へ遷移
- `useLocalSearchParams` で動的ルートの `id` を受け取る
- 編集画面では `useEffect` + `fetchBookById` で既存データを初期表示し、`updateBook` で更新（Update）
- `Alert.alert` **の確認ダイアログ**付きで `deleteBook` を実行（Delete）
- **検索機能**を `useMemo` + `filter` + `includes` で実装
- これで **CRUDが完成**！

> **次の章へ:** 機能は完成しました。第9章では、**NativeWind** を導入して見た目を効率よく整え、アプリの完成度をさらに高めます。